
Machine Learning Based Analog Circuit Fault Diagnosis
PDF 21 — Chapter 28: day16_confusion_analysis.py Explained Line-by-Line
The Detective Story in Code | How the Investigation Led to DC Level

PART 1 — The Error Analysis Engineering Workflow

Chapter 28 — Complete Python Source Code Explained Part 1 — day16_confusion_analysis.py (Based on your actual source code)

Before Starting

This chapter is probably the most important engineering chapter in your entire project. Why? Because this is where you stopped being a programmer and started thinking like an engineer. Most students would have done this.

Low Accuracy

Change Algorithm

Train Again

You didn't. Instead, you asked a much better question. Why is the model making mistakes? That single question changed the entire project. Goal of This Script Professor asks What is the purpose of day16_confusion_analysis.py?

Perfect Answer

This script performs a detailed error analysis of the trained classifier by examining the confusion matrix, classification report, and feature distributions. Its purpose is to identify systematic misclassifications and guide engineering improvements rather than blindly changing the machine learning algorithm.

The Engineering Workflow

Notice how your project evolved.

Train Model

Evaluate

Find Errors

Understand Errors

Improve Features

Train Again

This is exactly how real machine learning projects are developed.

Why Error Analysis Matters

Suppose your model achieves 95% accuracy. Question. Is it perfect? No. That remaining 5% contains valuable information. Instead of asking "How many mistakes?" we ask "Which mistakes?" That is the purpose of this script. Reading the Results Your code loads the trained model and evaluates it again to generate the confusion matrix and classification report. Question. Why repeat the evaluation? Because this script is not training. It is investigating.

Confusion Matrix

Your code creates the confusion matrix again. Conceptually, it looks like Predicted

N O S D Noise

Actual

N 40 0 0 1 0

O 3 36 1 0 0

S 0 1 39 0 0

D 2 0 0 38 0

Noise 0 0 0 1 39

Diagonal — Correct.

Off-diagonal — Mistakes.

But Your Script Goes Further

Most students stop here. Your script doesn't. It starts asking questions. Question 1 Which Class Performs Worst? Your code examines the classification report. Instead of looking only at accuracy, it compares Precision, Recall, and

F1 Score

for every class. Question. Why? Suppose overall accuracy is 96%. But one fault class has only 70% recall. That class needs attention, even though overall accuracy looks excellent.

PROFESSOR ASKS

Why examine class-wise metrics?

EXCELLENT ANSWER

Overall accuracy may hide poor performance on individual fault classes. Class-wise metrics identify which specific faults require further investigation and improvement. Question 2 Which Classes Are Being Confused? Now your code looks at the off-diagonal entries of the confusion matrix. Instead of saying "There are errors." It says "Exactly which classes are confused?" Example **RC_Open** — Predicted as. RC_Normal This is far more useful.

The Big Discovery

Your analysis revealed that RC_Open was the most problematic fault class. It wasn't a random mistake. It happened repeatedly. This means there was a systematic problem, not random noise.

PROFESSOR ASKS

What was the most important finding from the confusion matrix?

EXCELLENT ANSWER

The confusion matrix revealed that RC_Open faults were consistently misclassified, indicating a systematic weakness in the feature representation rather than a random classification error. Random Error vs Systematic Error Very important.

Random Error

Different mistakes everywhere.

Systematic Error

The same mistake repeats again and again. Question. Which one did your project have? Systematic. That means the model was missing important information.

Engineering Thinking

Many people would say Let's try Random Forest. Or Let's use XGBoost. You didn't. Instead, you asked Why is RC_Open difficult? That question is the reason your project became much stronger.

Ranking Misclassifications

Your code does something very interesting. It effectively identifies the largest off-diagonal values. Conceptually, RC_Open → Normal

Highest Confusion

RC_Drift → Normal

Second Highest

. Question. Why rank them? Because the largest errors deserve the highest priority. This is engineering prioritization. Why Not Improve Every Class? Suppose 10 classes exist. Only one class is causing most errors. Question. Should you redesign the entire system? No. Fix the biggest problem first. That is exactly what your project did.

Feature Comparison

Now your script starts comparing the features of confused classes. Instead of looking at the classifier, it looks at the data. Question. Are RC_Normal and RC_Open actually different in terms of the existing features? This investigation led to the biggest discovery in the project.

The Missing Information

After comparing the features, you realized something important. Most features described the transient response. Very few described the final steady-state voltage. That missing information explained why RC_Open looked similar to Normal under the original feature set.

The Detective Story

Your project didn't discover DC Level by luck. It followed a logical sequence.

Confusion Matrix

Repeated RC_Open Errors

Compare Features

Identify Missing Information

Add DC Level

Retrain Model

This is exactly how engineering research should work.

Why This Chapter Matters

This chapter is not about Python. It is not about SVM. It is about problem solving. It shows that your improvements were driven by evidence, not guesswork.

COMMON MISTAKES

MISTAKE: [X] Looking only at Accuracy. Wrong.

MISTAKE: [X] Ignoring repeated errors. Wrong. Repeated errors usually indicate missing information.

MISTAKE: [X] Changing algorithms before understanding the data. Wrong.

MISTAKE: [X] Assuming every misclassification is random. Wrong. Many are systematic and can be fixed.

REVISION NOTES

Remember the investigation process. **Confusion Matrix** — Find Weakest Class. **Identify Repeated Confusion** — Compare Features. **Find Missing Information** — Design Better Features. Memory Trick Remember this sentence. "Don't fix the model first. Understand the mistakes first." That sentence summarizes the philosophy behind Day 16.

KNOWLEDGE CHECK

Level 1

- What is the purpose of error analysis?
- Why is the confusion matrix more useful than accuracy?
- Which fault class caused the biggest problem?
- What is a systematic error?
- Why compare feature values after finding a confused class?

Level 2

- Explain how the confusion matrix guided feature engineering.
- Why is repeated misclassification more important than isolated errors?
- Why shouldn't the algorithm be changed immediately?
- How did feature comparison help identify missing information?
- Describe the engineering workflow followed in your project.

Professor Level

- Explain how Day 16 changed the direction of your project.
- Why is evidence-based feature engineering better than trial-and-error model changes?
- How did the confusion matrix lead to the discovery of DC Level?
- What distinguishes an engineering investigation from simple model tuning?
- If another fault class became problematic, how would you analyze it?
- End of Chapter 28 (Part 1)
- Congratulations.
- You now understand the most important engineering lesson in your project:
- The confusion matrix was not the end of the analysis—it was the beginning.
- Instead of treating errors as failures, you treated them as clues. That mindset led directly to the addition of DC Level and later the Topology Rule, which became the defining improvements of your project.

Author's Note The next continuation will cover Chapter 28 (Part 2): Feature-by-feature comparison between RC_Normal and RC_Open. Why the first 16 features were insufficient. The reasoning behind introducing DC Level. Visual interpretation of feature distributions. How you proved that DC Level solved the specific weakness instead of simply increasing the feature count. The engineering methodology behind feature selection. This chapter explains why Feature 17 exists, making it one of the most important parts of the Project Bible.

PART 2 — The Investigation That Led to DC Level

Chapter 28 — Complete Python Source Code Explained Part 2 — The Investigation That Led to DC Level (Based on your actual source code and analysis notebooks)

Before Starting

This is the chapter that separates your project from an ordinary machine learning project. Why? Because you did not add DC Level randomly. You proved it was needed. That is engineering.

Where We Left Off

The confusion matrix showed this. RC_Open

Repeatedly Misclassified

Question. What should we do next? Many students would answer Add another algorithm. Wrong. The correct answer is Investigate why.

Step 1 — Forming a Hypothesis

Every engineering investigation starts with a hypothesis. Your hypothesis was "Maybe the current feature set is missing some important physical information." Notice you did not assume the SVM was bad. You questioned the input, not the algorithm.

Step 2 — Comparing the Two Classes

Now your analysis focused only on **RC_Normal** — RC_Open. Not every class. Only the problematic pair. This makes the investigation much easier. Why Compare Only These Two? Question. Why not compare every fault? Because the confusion matrix already told us where the problem was. Engineering means solving the biggest problem first. Looking at Existing Features You examined the values of all existing features. Examples Peak RMS Mean

Rise Time

Settling Time

Signal Energy

. Question. What were you looking for? You wanted to know whether one feature clearly separated Normal from RC_Open. What Did You Discover? You found something interesting. Many features looked very similar. Example Conceptually Peak **Normal** — 4.98. **Open** — 4.95. **Difference** — Very small. Another example RMS **Normal** — 3.41. **Open** — 3.39. Again, very similar. Question. If the features are almost identical, can the SVM easily separate them? No. Because

Machine Learning

only knows the features you provide.

The Key Insight

Now think about the actual circuit. An RC circuit does not only have a transient. Eventually, it reaches steady state. Question. Were we explicitly measuring that steady-state voltage? No. That information was hidden inside the waveform, but not inside the feature vector. That was the missing piece.

The Engineering Thought Process

Notice the logic. Waveform

Contains Steady-State

Feature Vector

Does NOT explicitly contain Steady-State — Classifier struggles. Need a new feature That is how DC Level was born. Not by guessing. By reasoning. Why Not Use Mean Voltage? Good question. Someone may ask "You already have Mean. Why not use that?" Answer. Because Mean uses all 200 samples. Example Charging Charging Charging Stable Stable Stable The charging region changes the average. But we don't want the average. We want the final steady-state voltage. These are different

things.

Visual Example

Suppose

Waveform A

0 1 2 3 4 5 5 5 5 5 **Mean** — 3.5. **DC Level** — 5.

Waveform B

0 1 2 3 4 4.6 4.6 4.6 4.6 4.6 **Mean** — 3.24. **DC Level** — 4.6. Notice something. The difference between steady-state voltages is much clearer than the difference between means. That makes DC Level a stronger feature.

PROFESSOR ASKS
Why wasn't Mean Voltage sufficient?
EXCELLENT ANSWER

Mean Voltage averages the entire transient response, including the charging phase. DC Level focuses only on the steady-state region, which better captures the final operating condition of the circuit and provides stronger discrimination between similar fault classes. Why Use the Last 20 Samples? Question. Why not last 5? Or last 100? Because the last 20 samples represent a good balance. **Too few samples** — Sensitive. to random variation. **Too many samples** — Start including. the transient region. Twenty samples provide a stable estimate of the steady-state voltage.

Scientific Thinking

Notice how you improved the model. You did not say "Let's add random features." Instead, you asked "What physical property is missing?" That question is exactly what researchers do. Testing the Hypothesis After adding DC Level, you retrained the classifier. Question. Why? Because a hypothesis must be tested. Engineering is not about ideas. It is about verified improvements.

The Result

After retraining, the confusion between RC_Normal and RC_Open reduced significantly. The new feature provided the missing information the classifier needed.

PROFESSOR ASKS
How did you verify that DC Level was useful?
EXCELLENT ANSWER

After introducing the DC Level feature, the classifier was retrained and evaluated again. The confusion matrix showed a clear reduction in RC_Open misclassification, confirming that the new feature successfully captured previously missing information.

Why This Is Better Than Adding More Algorithms

Imagine you had tried

Random Forest,

XGBoost, and Neural Networks. Would they automatically know the steady-state voltage? No. They only learn from the features you provide. Better features often improve multiple algorithms, whereas changing algorithms cannot recover missing information.

Engineering Principle

Remember this forever. A better feature is often more valuable than a more complex algorithm. Your project is a perfect example of this principle. Complete Investigation Pipeline **Confusion Matrix** —

RC_Open Repeated Errors.

Compare Existing Features

Identify Missing Steady-State Information

Design DC Level Feature

Retrain Model

Reduced Confusion

Validated Improvement

This is one of the strongest stories you can tell during an interview.

COMMON MISTAKES

MISTAKE: [X] Adding features without a reason. Wrong.

MISTAKE: [X] Assuming more features always help. Wrong. Useful features help. Random features may hurt.

MISTAKE: [X] Treating feature engineering as trial and error. Wrong. It should be guided by evidence.

REVISION NOTES

Remember the logic. **Repeated Error** — Investigate. **Find Missing Information** — Design New Feature. Validate Improvement This is the scientific method applied to machine learning.

MEMORY TRICK

Remember this sentence. "The confusion matrix asked the question. DC Level answered it." That one sentence captures the entire engineering story behind

Feature 17.

KNOWLEDGE CHECK

Level 1

- Why was RC_Open difficult to classify?
- Why wasn't Mean Voltage enough?
- What does DC Level measure?
- Why use only the last 20 samples?
- How was the usefulness of DC Level verified?

Level 2

- Explain the complete reasoning that led to DC Level.
- Why is evidence-driven feature engineering important?
- Why can a better feature outperform a more complex algorithm?
- How did the confusion matrix guide the investigation?
- Describe the complete validation process for the new feature.

Professor Level

- Explain the engineering methodology that led to Feature 17.
- Why is steady-state information important in analog fault diagnosis?
- How did you prove that DC Level solved the RC_Open problem?
- If DC Level had not improved the results, what would your next step have been?
- Explain why your feature engineering process follows the scientific method.
- End of Chapter 28 (Part 2)
- Congratulations.
- You now understand why DC Level exists, not just how it was implemented.
- This is one of the strongest parts of your project because it demonstrates a complete engineering cycle:
 - Observe.
 - Investigate.
 - Form a hypothesis.
 - Design a solution.
 - Validate the result.
- That workflow is exactly what interviewers, professors, and research reviewers look for.

Author's Note The next continuation begins Chapter 29 — The Topology Rule. This chapter explains the second major engineering innovation in your project: Why DC Level alone wasn't the end of the story. The physical intuition behind the Topology Rule. Why combining machine learning with circuit knowledge is more powerful than using either alone. The complete implementation of the rule in your dashboard. Why this hybrid AI + domain knowledge approach makes your project significantly stronger than a standard SVM classifier.